A Review of Hybrid Encryption Techniques for Secure Cloud File Storage and Sharing: Identifying Gaps in Client-Side Key Management and Zero-Knowledge Architectures

Ashish Kumar, Ashish Singh, Avee Ranjan, Ayshi Shannidhya Panda
Under the guidance of **Mrs. Snehanjali Pradhan**
Department of Computer Science and Engineering
Silicon Institute of Technology, Bhubaneswar
PID: 23CSEP03

***Abstract*—Cloud storage services have become integral to modern data management, yet they present significant security challenges related to data confidentiality, key management, and secure multi-user file sharing. Hybrid encryption techniques combining symmetric algorithms (AES) with asymmetric algorithms (RSA) have emerged as the dominant approach for balancing performance and security. This paper reviews existing hybrid encryption architectures for cloud file storage, analyzing their approaches to encryption placement (client-side vs. server-side), key management models, and file-sharing mechanisms. Through a systematic comparison of recent literature (2022–2025), we identify a critical research gap: no existing system simultaneously provides true client-side encryption before upload, browser-only private key management, and zero-effort file sharing without requiring the server to access plaintext data or encryption keys. We propose a system architecture that addresses this gap through client-mediated key re-wrapping, browser-native cryptographic operations via the Web Crypto API, and an encrypted key export mechanism for key recovery.**

***Index Terms*—Hybrid Encryption, AES-256-GCM, RSA-OAEP, Cloud Storage Security, Client-Side Encryption, Web Crypto API, Key Envelope, Zero-Knowledge Architecture, File Sharing**

I. Introduction

Cloud computing has revolutionized data storage and access paradigms, enabling users to store, retrieve, and share files from any location through web-based interfaces [1]. However, this convenience introduces critical security concerns: data stored on third-party servers is vulnerable to unauthorized access by malicious actors, insider threats from cloud service providers, and legal compulsion through government subpoenas [2].

Traditional approaches to cloud data security rely on the cloud provider's built-in encryption mechanisms, where the provider manages both the encryption process and the keys. This model creates a fundamental trust problem — the entity storing the data also controls the keys to decrypt it [3]. A compromised cloud server or a malicious administrator can access all stored data, regardless of the encryption applied.

Hybrid encryption has emerged as the predominant solution for secure cloud storage [4]. By combining the computational efficiency of symmetric encryption (AES) for bulk data encryption with the secure key distribution properties of asymmetric encryption (RSA), hybrid schemes achieve both performance and security. The symmetric key used to encrypt each file (Data Encryption Key, or DEK) is itself encrypted using the user's RSA public key, creating a "key envelope" that can be safely stored alongside the ciphertext [5].

Despite significant progress, existing hybrid encryption systems for cloud storage face unresolved challenges in three interconnected areas: (1) the placement of encryption operations (client-side vs. server-side), (2) the management of private keys in browser environments, and (3) the mechanism for secure multi-user file sharing without compromising zero-knowledge guarantees.

This paper systematically reviews the current state of hybrid encryption techniques for cloud file storage, identifies the critical research gaps, and proposes architectural solutions to address them.

II. Literature Review

A. Symmetric Encryption for Cloud Storage

The Advanced Encryption Standard (AES) is the most widely adopted symmetric encryption algorithm for cloud data protection [6]. AES operates on fixed-size 128-bit blocks with key sizes of 128, 192, or 256 bits. For cloud storage applications, AES-256 is preferred due to its resistance against brute-force attacks.

The choice of block cipher mode significantly impacts both security and functionality:

- **AES-CBC (Cipher Block Chaining):** Widely used in legacy systems but provides only confidentiality without built-in integrity verification. Vulnerable to padding oracle attacks if improperly implemented [7].
- **AES-GCM (Galois/Counter Mode):** Provides authenticated encryption — both confidentiality and data integrity

through a built-in authentication tag. Recommended by NIST for modern applications [8]. Eliminates the need for separate HMAC computation.

Kumar et al. [9] demonstrated that AES-256-GCM achieves near-instantaneous encryption for files under 50 MB with negligible overhead from the authentication tag computation. Their work confirmed AES-GCM as the optimal choice for web-based cloud storage systems where both speed and integrity are required.

B. Asymmetric Encryption for Key Management

RSA (Rivest–Shamir–Adleman) remains the most widely deployed asymmetric algorithm for key exchange in cloud storage systems [10]. In hybrid encryption architectures, RSA is not used to encrypt files directly (due to computational overhead and message size limitations) but rather to encrypt the symmetric AES key.

Two RSA padding schemes are commonly employed:

- **PKCS#1 v1.5:** The legacy padding scheme, still found in many implementations. Known to be vulnerable to Bleichenbacher's chosen-ciphertext attack under certain conditions [11].
- **RSA-OAEP (Optimal Asymmetric Encryption Padding):** The recommended padding scheme, providing provable security under the RSA assumption. Uses SHA-256 for hashing and MGF1 for mask generation [12].

Patil et al. [14] emphasized that RSA-OAEP with SHA-256 should be the standard for new implementations, as PKCS#1 v1.5 has known vulnerabilities that can be exploited in automated attacks.

C. Hybrid Encryption Architectures

The hybrid encryption model for cloud storage follows a consistent pattern across the literature:

1) Generate a random AES session key (DEK) for each file
2) Encrypt the file using AES with the DEK
3) Encrypt the DEK using the recipient's RSA public key
4) Store the encrypted file and the encrypted DEK (key envelope) on the cloud server

Hassan et al. [13] implemented this model with AES-256-CBC and RSA-2048, achieving encryption throughputs of 150 MB/s for file data while maintaining key wrapping overhead under 5 ms. However, their system performed all encryption on the server side, meaning the server had access to plaintext files during the encryption process.

Singh and Mehta [4] proposed a client-side encryption variant where the browser encrypts files before upload. Their implementation used the Web Crypto API for AES-GCM encryption but relied on server-managed RSA keys, negating some benefits of client-side processing.

D. Client-Side Encryption and Browser-Based Key Management

The W3C Web Crypto API [17] provides native browser support for cryptographic operations including key generation, encryption, decryption, and digital signatures. This API enables client-side encryption without requiring external libraries or browser plugins.

Recent studies [14], [15] have explored browser-based key management using IndexedDB for persistent storage of cryptographic keys. The key advantage is that the user's RSA private key never leaves the browser, creating a zero-knowledge architecture where the server cannot decrypt user data even if compromised.

However, browser-based key storage introduces a critical vulnerability: clearing browser data, switching browsers, or formatting the device results in permanent key loss with no recovery mechanism [14].

E. Secure File Sharing Mechanisms

Multi-user file sharing in encrypted cloud storage requires re-encrypting the file's AES key for each recipient without re-encrypting the entire file. Three approaches exist in the literature:

- **Server-Mediated Sharing:** The server holds a copy of the AES key (encrypted with the server's public key). During sharing, the server decrypts the AES key and re-encrypts it with the recipient's public key [13]. This is simple but requires trusting the server with key material.
- **Owner-Mediated Sharing:** The file owner's client decrypts the AES key and re-encrypts it for the recipient. The server never sees the plaintext key [14]. This preserves zero-knowledge guarantees but requires the owner to be online.
- **Proxy Re-Encryption (PRE):** A cryptographic technique where a semi-trusted proxy can transform a ciphertext encrypted under one key into a ciphertext encrypted under another key, without accessing the plaintext [16]. Theoretically optimal but computationally expensive and complex to implement in browser environments.

III. Comparative Analysis

Table I presents a systematic comparison of recent hybrid encryption systems for cloud file storage across six critical dimensions.

The analysis reveals that no existing system achieves all three goals simultaneously: (1) true client-side encryption before upload, (2) browser-only private key management with recovery, and (3) seamless file sharing without server access to plaintext keys.

IV. Identified Research Gap

Based on the comprehensive review, we identify the following critical research gap:

A. The Encryption–Sharing Trade-off

Existing systems face a fundamental trade-off between encryption placement and sharing convenience:

- **Server-side encryption systems** [9], [13] achieve easy sharing (the server can re-wrap keys for recipients) but violate zero-knowledge guarantees because the server

TABLE I
COMPARATIVE ANALYSIS OF HYBRID ENCRYPTION SYSTEMS FOR CLOUD FILE STORAGE (2022–2025)

Feature	**Hassan et al. (2023)**	**Singh & Mehta (2023)**	**Kumar et al. (2024)**	**Patil et al. (2024)**	**Nimbus Cloud (2024)**	**CloudVault (Proposed)**
Encryption Algorithm	AES-256-CBC	AES-256-GCM	AES-256-GCM	AES-256-GCM	AES-256-GCM	AES-256-GCM
Key Wrapping	RSA-2048 PKCS#1	RSA-2048 OAEP	RSA-2048 OAEP	RSA-2048 OAEP	RSA-2048 OAEP	RSA-2048 OAEP
Encryption Location	Server-side	Client-side	Server-side	Client-side	Client-side	Client-side
Private Key Storage	Server DB	Server-managed	Server DB	IndexedDB	IndexedDB	IndexedDB
Sharing Mechanism	Server-mediated	Not implemented	Server-mediated	Owner-mediated	Not implemented	Client-mediated
Key Recovery	Password reset	Password-based	No	No	No	Encrypted export
Per-User Key Entries	No	No	No	Per-file only	Per-file only	Per-user per-file
Working Implementation	Partial	Partial	Theoretical	Partial	Yes	Yes

sees plaintext files during upload and holds copies of encryption keys.

- **Client-side encryption systems** [4], [14] preserve zero-knowledge properties but either do not implement sharing, or require complex owner-involvement protocols that reduce usability.

No existing work provides a practical, implemented system that achieves **both** true client-side encryption **and** zero-effort file sharing without server access to plaintext data or keys.

B. Browser Key Recovery Gap

Systems that store RSA private keys exclusively in browser storage (IndexedDB) provide strong security guarantees but offer no recovery mechanism. If the browser data is cleared, the user permanently loses access to all their files. Existing literature acknowledges this limitation but does not propose practical solutions compatible with zero-knowledge architectures [14].

C. Absence of Per-User Key Envelope Architecture

Most reviewed systems store a single encrypted key per file. This limits access control granularity — sharing requires either re-encrypting the file itself or maintaining complex access control lists. A per-user, per-file key envelope architecture (where each authorized user has their own encrypted copy of the AES key) enables fine-grained access control through simple key entry creation and deletion, but has not been fully explored in existing literature.

V. PROPOSED SYSTEM ARCHITECTURE

To address the identified gaps, we propose **CloudVault** — a cloud-based file security system with the following architectural innovations:

A. Client-Side Encryption Before Upload

The browser generates a random AES-256-GCM key and encrypts the file entirely in the client using the Web Crypto API before any data is transmitted to the server. The server receives and stores only ciphertext, ensuring it never accesses plaintext file data.

B. Browser-Only RSA Key Management

RSA-2048-OAEP key pairs are generated in the browser using the Web Crypto API. The private key is stored exclusively in IndexedDB and is never transmitted to the server. Only the public key is sent during registration.

C. Client-Mediated Key Re-Wrapping for Sharing

When the file owner shares a file, the owner's browser:

1) Retrieves its own encrypted AES key (OWNER entry) from the server
2) Decrypts it using the private key from IndexedDB
3) Fetches the recipient's RSA public key from the server
4) Re-encrypts the AES key with the recipient's public key
5) Sends the new SHARED key entry to the server

The server facilitates the exchange but **never** sees the plaintext AES key. This eliminates the need for a SERVER key entry entirely.

D. Key Envelope Table with Per-User Entries

Each file maintains a Key Envelope Table with entries of type OWNER and SHARED. Each entry contains the file's AES key encrypted with a specific user's RSA public key. Access control is managed through key entry creation (sharing) and deletion (revocation).

E. Encrypted Key Export for Recovery

To address the key recovery gap, the system provides an encrypted key export mechanism:

1) The private key is exported from IndexedDB

2) A passphrase is derived into an AES key using PBKDF2
3) The private key is encrypted with the derived AES key
4) The encrypted bundle is downloaded as a backup file

Recovery follows the reverse process: import the backup file, enter the passphrase, and restore the private key to IndexedDB.

VI. Problem Statement

Existing cloud storage systems employing hybrid AES-RSA encryption face a fundamental trade-off between security and usability in multi-user file sharing. Server-side encryption models compromise zero-knowledge guarantees by exposing plaintext data to the server during upload, while client-side encryption models introduce complex key management overhead that hampers seamless file sharing. Furthermore, browser-based private key storage lacks recovery mechanisms, making key loss equivalent to permanent data loss.

This work proposes a cloud file security system that achieves true end-to-end encryption with client-side AES-256-GCM encryption before upload, browser-only RSA key management via the Web Crypto API and IndexedDB, and a novel client-mediated key re-wrapping mechanism for zero-effort file sharing — eliminating the need for the server to access plaintext data or encryption keys at any stage.

VII. Conclusion

This review paper has systematically analyzed existing hybrid encryption techniques for secure cloud file storage and sharing. Through a comparative study of recent implementations (2022–2025), we identified a critical research gap: the absence of a practical system that simultaneously achieves true client-side encryption, browser-only key management with recovery, and zero-effort file sharing without server access to plaintext data or keys.

The proposed CloudVault architecture addresses this gap through three key innovations: (1) client-side AES-256-GCM encryption before upload using the Web Crypto API, (2) a client-mediated key re-wrapping mechanism that eliminates the need for server-held key copies, and (3) an encrypted key export mechanism for secure private key backup and recovery. The per-user Key Envelope Table design enables fine-grained access control through simple key entry management.

Future work will focus on implementing the proposed fixes, conducting performance benchmarking against existing systems, and exploring post-quantum cryptographic alternatives (such as lattice-based key encapsulation mechanisms) to future-proof the system against quantum computing threats.

References

[1] P. Mell and T. Grance, "The NIST Definition of Cloud Computing," *NIST Special Publication 800-145*, 2011.

[2] D. Chen and H. Zhao, "Data Security and Privacy Protection Issues in Cloud Computing," in *Proc. IEEE International Conference on Computer Science and Electronics Engineering (ICCSEE)*, 2012, pp. 647–651.

[3] S. Kamara and K. Lauter, "Cryptographic Cloud Storage," in *Financial Cryptography and Data Security*, Springer, 2010, pp. 136–149.

[4] A. Singh and R. Mehta, "Hybrid AES-RSA Encryption Model for Secure Cloud Data Storage with Client-Side Processing," *International Journal of Computer Applications*, vol. 185, no. 12, pp. 34–42, 2023.

[5] E. Barker, "Recommendation for Key Management: Part 1 – General," *NIST Special Publication 800-57*, Rev. 5, 2020.

[6] J. Daemen and V. Rijmen, *The Design of Rijndael: AES – The Advanced Encryption Standard*. Springer, 2002.

[7] S. Vaudenay, "Security Flaws Induced by CBC Padding," in *Advances in Cryptology – EUROCRYPT 2002*, Springer, 2002, pp. 534–546.

[8] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," *NIST Special Publication 800-38D*, 2007.

[9] R. Kumar, S. Verma, and P. Gupta, "Hybrid AES-RSA Encryption for Secure Cloud Storage and File Sharing," *International Journal of Intelligent Systems and Applications in Engineering (IJISAE)*, vol. 12, no. 4, pp. 112–123, 2024.

[10] R. L. Rivest, A. Shamir, and L. Adleman, "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.

[11] D. Bleichenbacher, "Chosen Ciphertext Attacks Against Protocols Based on the RSA Encryption Standard PKCS #1," in *Advances in Cryptology – CRYPTO '98*, Springer, 1998, pp. 1–12.

[12] M. Bellare and P. Rogaway, "Optimal Asymmetric Encryption," in *Advances in Cryptology – EUROCRYPT '94*, Springer, 1994, pp. 92–111.

[13] M. Hassan, A. Ali, and F. Khan, "Enhanced Cloud Data Security Using Hybrid Cryptographic Techniques," in *Proc. IEEE International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, 2023, pp. 1–7.

[14] S. Patil, R. Deshmukh, and V. Joshi, "Client-Side Encryption Based Secure Data Sharing Schemes for Cloud Platforms," *ResearchGate Preprint*, 2024, doi:10.13140/RG.2.2.xxxxx.

[15] "Nimbus Cloud: Browser-Based Secure Cloud Storage with Client-Side Encryption," *International Journal of Science, Engineering and Technology (IJSET)*, vol. 12, no. 2, 2024.

[16] G. Ateniese, K. Fu, M. Green, and S. Hohenberger, "Improved Proxy Re-Encryption Schemes with Applications to Secure Distributed Storage," *ACM Transactions on Information and System Security*, vol. 9, no. 1, pp. 1–30, 2006.

[17] W3C, "Web Cryptography API," W3C Recommendation, 2017. [Online]. Available: https://www.w3.org/TR/WebCryptoAPI/